Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Sector No. 26, Pradhikaran, Nigdi, Pune, 411044

An Autonomous Institute Approved By AICTE and Affiliated To SPPU, Pune

DEPARTMENT OF INFORMATION TECHNOLOGY



Cryptography and Network Security (BIT27PC01)

Final Year B.Tech

Academic Year : 2026 – 27

Semester – VII

Formative Assessment I & II

Submitted By A4

Secure Hospital Patient Records System: A Hybrid Cryptographic Framework Using AES-256-GCM, RSA, SHA-256 and Role-Based Access Control

Guide: Dr. Jayashree V. Katti, Head of IT Department

Team Member	PRN
Shivanjali Bhosale	123B1F009
Rutuja Bobade	123B1F012
Aditya Deore	123B1F019
Chaitanya Deore	123B1F020

1. Executive Summary

Modern healthcare facilities exchange confidential medical reports, surgical evaluations, lab panels, and radiological findings across interconnected clinical specialties. Standard Electronic Health Record (EHR) systems frequently maintain patient records in unencrypted database tables or rely exclusively on perimeter transport encryption (TLS). Consequently, databases remain vulnerable to insider tampering, SQL injection exfiltration, Man-In-The-Middle (MITM) diagnostic parameter alteration, and prescription forgery.

SecureMed is an end-to-end zero-knowledge clinical report exchange platform. The central application server acts strictly as an untrusted, zero-knowledge courier: plaintext medical reports and plaintext private keys are never stored on disk or in the database. Security is mathematically guaranteed through SHA-256 digests, RSA-PSS 2048-bit digital signatures, AES-256-GCM envelope encryption, multi-recipient RSA-OAEP key wrapping, an internal X.509 Root Certificate Authority (CA) with a live Certificate Revocation List (CRL), anti-replay nonces with sliding freshness windows, and a blockchain-style SHA-256 hash-chained audit ledger.

2. Problem Statement & STRIDE Threat Model

In clinical environments, the compromise of medical information leads to direct patient harm. Intercepted reports violate HIPAA/GDPR privacy mandates, while unauthorized alteration of clinical diagnostic values (e.g., blood potassium levels, biopsy indications, chemotherapy dosages) can cause fatal clinical misdiagnoses.

To address these threats rigorously, SecureMed implements countermeasures aligned with the STRIDE security framework:

- **Spoofing Defense:** Sender identity is tied to an X.509 v3 certificate issued by the hospital's internal Root CA. Senders must generate an RSA-PSS digital signature over the report hash; recipients verify signatures strictly against the sender's public key certificate.
- **Tampering Defense:** Payloads are encrypted using AES-256-GCM, producing a 128-bit Poly1305/GHASH authentication tag. Any single-bit alteration halts decryption and triggers an immediate tampered status.
- **Repudiation Defense:** Reports are signed with the clinician's password-derived private key. Because private keys are held exclusively by the respective clinician, mathematical non-repudiation is enforced.
- **Information Disclosure Defense:** Plaintext reports are never written to disk or the database. Payloads exist solely as AES-256-GCM binary ciphertext files, with session keys encapsulated via RSA-OAEP.
- **Denial of Service Defense:** Flask-Limiter rate-limiting, 16MB file upload caps, and automatic 15-minute account lockout after 5 failed login attempts prevent resource exhaustion.
- **Elevation of Privilege Defense:** Strict server-side Role-Based Access Control (RBAC) decorators gate all routes, combined with a data-driven Access Control Matrix regulating inter-department communication.

3. Cryptographic Algorithms & Mechanics

SecureMed exclusively employs industry-standard cryptographic primitives from the Python cryptography library:

Algorithm	Type / Standard	Specific Purpose	Execution Timing
Argon2id	Memory-Hard KDF	Password hashing resisting GPU/ASIC attacks	User registration & login verification
Scrypt	Memory-Hard KDF	Derives 256-bit key from password to encrypt private key	Account provisioning & session unlock
AES-256-GCM	Symmetric AEAD	Payload encryption with 128-	Report dispatch & recipient

		bit authentication tag	decryption
RSA-PSS	Asymmetric Signature	Digital signature with MGF1-SHA256 (Non-Repudiation)	Sender signing & receiver verification
RSA-OAEP	Key Encapsulation	Wraps AES session key separately for each recipient	Multi-recipient envelope creation
SHA-256	Cryptographic Hash	Payload digests & blockchain-style audit hashing	Pre-signing, verification, audit events
X.509 v3	PKI Public Cert	Binds user/department identity to public key	CA issuance & receiver trust verification
TOTP	Time-Based OTP	Two-Factor Authentication (RFC 6238)	Account login with authenticator apps

4. Multi-Department Architecture & Clinical Workflows

Hospital departments are dynamic database entities equipped with dedicated RSA-2048 keypairs, Root CA-signed X.509 certificates, and departmental member key-wrapping. The system seeds 9 foundational departments:

- **Clinical Specialties:** Outpatient (OPD), Pathology/Lab (PATH), Radiology (RAD), Pharmacy (PHARM), Surgery (SURG), Emergency/ICU (EMERG), Cardiology (CARD), Billing/Insurance (BILL), and Medical Records (HIM).
- **Runtime Provisioning:** Hospital administrators can add, update, or disable departments from the web interface without writing a single line of code. Dedicated asymmetric keypairs and certificates are issued automatically.
- **Department Key-Wrapping:** Reports addressed to an entire specialty (e.g., Pathology) are decrypted via an encapsulated department symmetric key wrapped for each authorized staff member.
- **Access Control Matrix:** Administrators define explicit inter-department exchange rules (e.g., Surgery can order from Pathology, but Billing cannot access raw surgical pathology notes).

5. Blockchain-Style Hash-Chained Audit Ledger

To prevent malicious administrators or intruders from tampering with forensic records, all system activities are recorded in a cryptographic hash chain:

H_i = SHA-256(H_{i-1} || Sequence || Timestamp || ActorID || Action || Target || Details)

- **Genesis Block:** The first entry links to a constant Genesis Hash of 64 zeros (H_0 = 0*64).
- **Tamper Evident:** Altering, deleting, or reordering any audit row invalidates the cryptographic hash chain for all subsequent entries.
- **Verification Engine:** The audit dashboard includes a 'Verify Chain Integrity' tool that recalculates every hash from Genesis to verify ledger authenticity in milliseconds.

6. Emergency Break-Glass Protocol & Patient Portal

In emergency trauma scenarios where patients are unconscious or unable to provide consent, clinical care cannot be halted. SecureMed incorporates an Emergency Break-Glass Override Protocol:

- **Break-Glass Workflow:** Attending physicians can bypass normal consent barriers by providing a mandatory clinical emergency justification (>= 15 characters).
- **Critical Alerting:** Break-glass events trigger high-priority alerts across the administrative security dashboard and append an immutable event to the audit ledger.

Patients access their own dedicated portal (/patients/portal) where they can view finalized reports, grant or revoke consent per doctor/department, and inspect the 'Who Viewed My Data' transparency ledger.

7. Security Lab & Verification Results

The platform features an in-app Security Lab (/demo/lab) allowing users and evaluators to execute live cryptographic attacks and observe real-time defenses:

- **Ciphertext Bit-Flipping:** Altering 1 bit in transit fails AES-256-GCM AEAD authentication and halts decryption.
- **Sender Forgery:** Forged signatures claiming to be another clinician are rejected mathematically under RSA-PSS.
- **Replay Retransmission:** Duplicated message envelopes are blocked by database nonce checks and the 300s freshness window.

The complete architecture has been validated via an automated pytest test suite across all 16 distinct test cases with a 100% pass rate:

- **Test Suite Coverage:** 16 passed in 18.62s covering password hashing, hybrid encryption, Root CA lifecycle, CRL revocation, department provisioning, hash-chained ledger, and patient consent.

8. Deployment & Execution

SecureMed is runnable on a standard laptop with a single command:

- **One-Command Execution:** python run.py automatically provisions the SQLite database, generates Root CA and dev TLS certificates, seeds demo accounts, and launches at https://127.0.0.1:5000.
- **Containerization:** Production Dockerfile and docker-compose.yml files enable rapid containerized deployment.